



# CSc 453

COMPILERS AND  
SYSTEMS  
SOFTWARE

Version: 2026-04-29

# FINALS REVIEW SPRING 2026

# Problem 1 (a)

Points: 10/100

- Assume a stack-based bytecode language with the operations on the next page.

1. Suppose you have written an indirect threaded interpreter. Give two techniques that can be used to speed it up.

| mnemoni | Op | stack<br>pre | stack<br>post | side-<br>effects                                                                    |
|---------|----|--------------|---------------|-------------------------------------------------------------------------------------|
| ADD     | 0  | [ A , B ]    | [ A+B ]       |                                                                                     |
| PUSHB X | 1  | [ ]          | [ X ]         |                                                                                     |
| PRINT   | 2  | [ A ]        | [ ]           | Print A                                                                             |
| BGE L   | 3  | [ A , B ]    | [ ]           | if A>=B then PC+=L                                                                  |
| BGE L   | 4  | [ A , B ]    | [ ]           | PC+=L                                                                               |
| LOAD X  | 5  | [ ]          | [ Mem[ X ] ]  | X is the variable's integer offset. The first variable has offset 0.                |
| STORE X | 6  | [ A ]        | [ ]           | Memory[X] = A; X is the variable's integer offset. The first variable has offset 0. |
| EXIT    | 7  |              |               | The interpreter exits                                                               |



# Problem 1 (c)

Points: 10/100

2. Show the bytecode that should be generated from the following Luca program:

```
PROGRAM P;  
VAR X : INTEGER;  
BEGIN  
    X := 1;  
    WHILE X < 10 DO  
        PRINT X;  
        X := X + 1;  
    END;  
END;
```

# Problem 1 (d)

Points: 10/100

3. Draw the Control-Flow Graph for the bytecode program in the previous problem.

# Problem 1 (e)

Points: 10/100

4. Write a direct-threaded interpreter for the bytecode language above. You should use gcc C as your main implementation language but you may resort to pseudo-code when that simplifies the coding. You don't have to provide implementations for `PUSHB`, `BRA`, `STORE`, `EXIT`.

- The next page has a template you can work from.



```
void engine() {  
    int Stack[100]; register int sp=0;  
    int Mem[100];  
    register Inst *pc =
```

```
    add:
```

```
    bge:
```

```
    load:
```

```
}
```

# Problem 2 (a)

Points: 10/100

- Give short (one-two sentence) explanations of these terms:
  - Register allocation
  - Register assignment
  - Register spilling
  - Instruction selection
  - Instruction scheduling
- Explain, in detail, what information a code generator stores in a *register table* and an *address table*. You may illustrate with an example.



# Problem 2 (b)

- Explain these terms:

1. HAS-A vs. IS-A

2. Late Binding

3. Inclusion Polymorphism

4. Runtime Type Checks

5. Dynamic Dispatch

# Problem 2 (c)

- Define these terms:
  1. Local optimization
  2. Global optimization
  3. Inter-procedural optimization
  4. Peephole optimization
  5. Semantics-preserving transformation
  6. Profile -guided code optimization
  7. Runtime optimization

# Problem 2 (c)

- Explain what each of these local code transformations do:
  1. Local common subexpression elimination.
  2. Local copy propagation.
  3. Local dead-code elimination.
  4. Algebraic optimization.
  5. Jump-to-jump removal.
  6. Reduction in strength.



# Problem 3 (a)

Points: 10/100

- In most languages that have a case/switch statement, it is illegal to use the same case-label more than once. Write a tree-walk evaluator that uses *threaded* attributes to check for multiply declared labels.
- You can assume that the programming language you're using has a built-in set type with the usual operations (member, union, intersection, etc).
- You can also assume that there exists a routine `ConstExpr(n)` which performs semantic analysis on the constant expression AST whose root is `n`, and returns its value in `n.Val`.

# Problem 3 (b)

- Here's an example case-statement from Modula-2:

```
CASE i OF
  5 : j := 66; |
  2 : j := 77; |
  5 : j := 99; |
  ELSE j := 0;
END
```

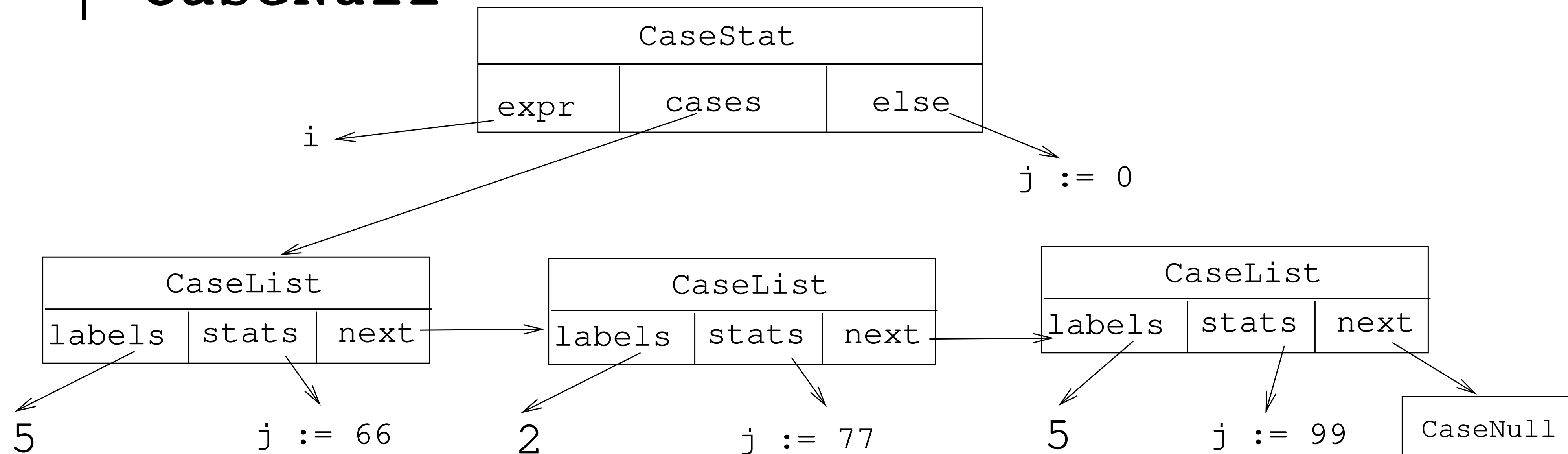
- Note that 5 occurs twice.

# Problem 3 (c)

- Here's the abstract syntax and the AST corresponding to the example above:

CaseStat: expr:Expr cases:CaseList else:Stat

CaseList: label:ConstExpr stats:Stat next:CaseList  
| CaseNull





```
void Stat (Node n) {  
    if (n.Kind == CaseStat) {  
  
        CaseList(n.cases);  
        /* n.cases.LabsOut available here! */  
  
    } else if (n.Kind == AssignStat) {  
    }  
}
```

```
void CaseList (Node n)  {
    if (n.Kind == CaseList) {
        ConstExpr(n.expr);
        /* Returns label in n.expr.Val */

    } else if (n.Kind == CaseNull) {

    }

}
```



# Problem 4

Points: 10/100

- Consider the Java program on the next page.

1. Explain in detail what should happen at

A. compile-time and at

B. runtime (if the code compiles)

for the 3 assignment statements in `main`. In particular, explain how the compiler and runtime system decide whether each assignment is legal or not.

2. Show the object layout for class D.

3. Show the code that the Java runtime system would execute in order to decide which method to call in the method invocation `c.foo()`. Draw a figure to illustrate.

```
class A {
    int x;
    public void foo(){}
}
class B extends A {
}
class C extends A {
    int y;
    public void foo(){}
}
class D extends C {
    int y;
}
class E extends C {
}
```

```
class Main {
    public static void main(
        String[] args) {
        A a=new A();
        B b=new B();
        C c=new C();
        D d=new D();
        E e=new E();
        b = (B)c;
        c = (C)e;
        c = (C)a;
        c.foo();
    }
}
```

# Problem 5

Points: 10/100

- Consider the Java program on the next 2 pages.

1. Draw the state of the Java virtual machine immediately before executing the last statement in main. Assume that no garbage collection occurs during the processing of main. Include in your picture all the objects that reside in memory: stacks, templates, heap objects, static globals, etc. You can ignore constructors. Draw all the pointers between these objects.

2. Explain, in detail, what happens when the statement `x = p3.getX();` gets executed.

3. Explain, in detail, what happens when the statement `p3 = (ColorPoint)p1;` gets executed.



```
class Point {
    int x; int y;
    public Point(int x, int y) {this.x = x; this.y = y;}
    public int getX(){return x;}
    public int getY(){return y;}
    public String toString(){return "[" + x + "," + y + "];"}
}

class Point3D extends Point {
    int z;
    public Point3D(int x, int y, int z) {super(x,y); this.z = z;}
    public int getZ(){return z;}
    public String toString(){return "[" + x + "," + y + "," + z + "];"}
}
```

```
class ColorPoint extends Point {
    int c;
    public ColorPoint(int x, int y, int c) {super(x,y); this.c = c;}
    public String getColor(){
        switch(c) {
            case 1: return "red";
            default: return "unknown";
        }
    }
    public String toString(){return "["+x+", "+y+", " + getColor() + "];"}
}

public class oo {
    static int x;
    public static void main(String args[]) {
        Point p1 = new Point(10,20);
        Point3D p2 = new Point3D(30,40,50);
        ColorPoint p3 = new ColorPoint(60,70,1);
        x = p3.getX();
        p3 = (ColorPoint)p1;
    }
}
```

# Problem 6

Points: 10/100

- Assume that we have an intermediate code language consisting of the following types of instructions:
  - $X := Y + Z$
  - `if  $X < Y$  then goto L`
  - `goto L`
- Give an algorithm for constructing control flow graphs from a list of such intermediate code instructions.





# DEFINITIONS AND ALGORITHMS

**COLLBERG.CS.ARIZONA.EDU**

**LIGERLABS.ORG**

**SUPPORTED BY**  
**NSF SATC/TTP-1525820**  
**SATC/EDU-2029632**

**COPYRIGHT © 2024-2026**

**CHRISTIAN COLLBERG**

**UNIVERSITY OF ARIZONA**